Java Practical Answer

Command for Terminal: (1. javac filename.java 2. java filename)

Assignment 1

Q1: Write a java program to accept names of ‘n’ cities, insert same into array list collection and display the contents of same array list, also remove all these elements.

```
import java.util.ArrayList;
import java.util.Scanner;
public class setA1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<String> cities = new ArrayList<>();
        System.out.print("Enter number of cities: ");
        int n = sc.nextInt();
        sc.nextLine();   // consume newline
        // Accept city names
        for (int i = 0; i < n; i++) {
            System.out.print("Enter city name: ");
            String city = sc.nextLine();
            cities.add(city);
        }
        // Display cities
        System.out.println("\nCities in ArrayList:");
        System.out.println(cities);
        // Remove all elements
        cities.clear();
        // Display after removal
        System.out.println("\nAfter removing all cities:");
        System.out.println(cities);
    }
}
```

Q2: Write a java program to read ‘n’ names of your friends, store it into linked list, also display contents of the same.

```
import java.util.LinkedList;
import java.util.Scanner;
public class setA2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        LinkedList<String> friends = new LinkedList<>();
        System.out.print("Enter number of friends: ");
        int n = sc.nextInt();
```

```
        sc.nextLine();   // consume newline
        // Accept friend names
        for (int i = 0; i < n; i++) {
            System.out.print("Enter friend name: ");
            String name = sc.nextLine();
            friends.add(name);
        }
        // Display LinkedList contents
        System.out.println("\nFriends in LinkedList:");
        System.out.println(friends);
    }
}
```

Q3: Write a program to create a new tree set, add some colors (string) and print out the tree set.

```
import java.util.TreeSet;
public class setA3 {
    public static void main(String[] args) {
        TreeSet<String> colors = new TreeSet<>();
        // Add colors
        colors.add("Red");
        colors.add("Blue");
        colors.add("Green");
        colors.add("Yellow");
        colors.add("White");
        // Display TreeSet
        System.out.println("Colors in TreeSet:");
        System.out.println(colors);
    }
}
```

Q4: Create the hash table that will maintain the mobile number and student name. Display the contact list.

```
import java.util.Hashtable;
import java.util.Enumeration;
public class setA4 {
    public static void main(String[] args) {
        Hashtable<String, String> contacts = new Hashtable<>();
        // Add mobile number and student name
        contacts.put("9876543210", "Amit");
        contacts.put("9123456780", "Rohit");
        contacts.put("9988776655", "Neha");
        contacts.put("9090909090", "Sneha");
        // Display contact list
        System.out.println("Student Contact List:");
        Enumeration<String> keys = contacts.keys();
```

```
        while (keys.hasMoreElements()) {
            String mobile = keys.nextElement();
            System.out.println("Mobile: " + mobile +
                               "  Name: " + contacts.get(mobile));
        }
    }
}
```

Q5: Accept ‘n’ integers from the user. Store and display integers in sorted order having proper collection class. The collection should not accept duplicate elements.

```
import java.util.Scanner;
import java.util.TreeSet;
public class setB1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        TreeSet<Integer> numbers = new TreeSet<>();
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++) {
            numbers.add(sc.nextInt());   // duplicates are ignored automatically
        }
        System.out.println("Sorted elements without duplicates:");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
        sc.close();
    }
}
```

Q6: Write a program to sort HashMap by keys and display the details before sorting and after sorting.

```
import java.util.HashMap;
import java.util.Map;
import java.util.TreeMap;
public class setB2 {
    public static void main(String[] args) {
        // Create HashMap
        HashMap<Integer, String> hm = new HashMap<>();
        hm.put(3, "Rahul");
        hm.put(1, "Amit");
        hm.put(4, "Suresh");
        hm.put(2, "Neha");
        // Display before sorting
        System.out.println("Before Sorting (HashMap):");
```

```
        for (Map.Entry<Integer, String> entry : hm.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
        // Sort HashMap by keys using TreeMap
        TreeMap<Integer, String> tm = new TreeMap<>(hm);
        // Display after sorting
        System.out.println("\nAfter Sorting by Keys (TreeMap):");
        for (Map.Entry<Integer, String> entry : tm.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
    }
}
```

Q7: Write a program that loads names and phone numbers from a text file where the data is organized as one line per record and each field in a record are separated by a tab (\t).it takes a name or phone number as input and prints the corresponding other value from the hash table (hint: use hash tables)

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.util.Hashtable;
import java.util.Scanner;
public class setB3 {
    public static void main(String[] args) {
        Hashtable<String, String> ht = new Hashtable<>();
        Scanner sc = new Scanner(System.in);
        try {
            BufferedReader br = new BufferedReader(new FileReader("data.txt"));
            String line;
            // Read file and store in Hashtable
            while ((line = br.readLine()) != null) {
                String[] data = line.split("\t");
                ht.put(data[0], data[1]);   // name as key, phone as value
            }
            br.close();
            // User input for search
            System.out.print("Enter name or phone number to search: ");
            String input = sc.nextLine();
            boolean found = false;
            // Search logic
            for (String name : ht.keySet()) {
                if (name.equalsIgnoreCase(input)) {
                    System.out.println("Phone Number: " + ht.get(name));
                    found = true;
                    break;
                } else if (ht.get(name).equals(input)) {
```

```
                    System.out.println("Name: " + name);
                    found = true;
                    break;
                }
            }
            if (!found) {
                System.out.println("Record not found");
            }
        } catch (Exception e) {
            System.out.println("Error reading file");
        }
        sc.close();
    }
}
```

Q8: Create a java application to store city names and their STD codes using an appropriate collection. The GUI should allow the following operations:

i. Add a new city and its code (No duplicates)

ii. Remove a city from the collection

iii. Search for a city name and display the code

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.HashMap;
public class setC1 extends JFrame implements ActionListener {
    JTextField cityField, stdField;
    JButton addBtn, removeBtn, searchBtn;
    JTextArea outputArea;
    HashMap<String, String> cityMap = new HashMap<>();
    setC1() {
        setTitle("City STD Code Manager");
        setSize(400, 350);
        setLayout(new FlowLayout());
        add(new JLabel("City Name:"));
        cityField = new JTextField(15);
        add(cityField);
        add(new JLabel("STD Code:"));
        stdField = new JTextField(15);
        add(stdField);
        addBtn = new JButton("Add City");
        removeBtn = new JButton("Remove City");
        searchBtn = new JButton("Search City");
```

```
        add(addBtn);
        add(removeBtn);
        add(searchBtn);
        outputArea = new JTextArea(10, 30);
        outputArea.setEditable(false);
        add(new JScrollPane(outputArea));
        addBtn.addActionListener(this);
        removeBtn.addActionListener(this);
        searchBtn.addActionListener(this);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }
    public void actionPerformed(ActionEvent e) {
        String city = cityField.getText().trim();
        String std = stdField.getText().trim();
        if (e.getSource() == addBtn) {
            if (cityMap.containsKey(city)) {
                outputArea.setText("City already exists!");
            } else {
                cityMap.put(city, std);
                outputArea.setText("City added successfully.");
            }
        }
        if (e.getSource() == removeBtn) {
            if (cityMap.remove(city) != null) {
                outputArea.setText("City removed successfully.");
            } else {
                outputArea.setText("City not found.");
            }
        }
        if (e.getSource() == searchBtn) {
            if (cityMap.containsKey(city)) {
                outputArea.setText("STD Code of " + city + " is " + cityMap.get(city));
            } else {
                outputArea.setText("City not found.");
            }
        }
    }
    public static void main(String[] args) {
        new setC1();
    }
}
```

Q9: Write a program to create link list of integer objects. Do the following:

i. add element at first position

ii. delete last element

iii. display the size of link list

```
import java.util.LinkedList;
public class setC2 {
    public static void main(String[] args) {
        LinkedList<Integer> list = new LinkedList<>();
        // Adding elements
        list.add(10);
        list.add(20);
        list.add(30);
        System.out.println("Initial Linked List: " + list);
        // i. Add element at first position
        list.addFirst(5);
        System.out.println("After adding at first position: " + list);
        // ii. Delete last element
        list.removeLast();
        System.out.println("After deleting last element: " + list);
        // iii. Display size of linked list
        System.out.println("Size of Linked List: " + list.size());
    }
}
```

Q10: Read a text file, specified by the first command line argument, into a list. The program should then display a menu which performs the following operations on the list:1. Insert line 2. Delete line 3. Append line 4. Modify line 5. Exit When the user selects Exit, save the contents of the list to the file and end the program.

```
import java.io.*;
import java.util.*;
public class setC3 {
    public static void main(String[] args) throws Exception {
        if (args.length == 0) {
            System.out.println("Please provide file name as command line argument.");
            return;
        }
        String filename = args[0];
        List<String> list = new ArrayList<>();
        Scanner sc = new Scanner(System.in);
        // Read file into list
        BufferedReader br = new BufferedReader(new FileReader(filename));
        String line;
        while ((line = br.readLine()) != null) {
            list.add(line);
```

```
}
br.close();
int choice;
do {
    System.out.println("\n---- MENU ----");
    System.out.println("1. Insert line");
    System.out.println("2. Delete line");
    System.out.println("3. Append line");
    System.out.println("4. Modify line");
    System.out.println("5. Exit");
    System.out.print("Enter choice: ");
    choice = sc.nextInt();
    sc.nextLine(); // consume newline
    switch (choice) {
        case 1:
            System.out.print("Enter position: ");
            int pos = sc.nextInt();
            sc.nextLine();
            System.out.print("Enter line: ");
            String newLine = sc.nextLine();
            list.add(pos - 1, newLine);
            break;
        case 2:
            System.out.print("Enter line number to delete: ");
            int del = sc.nextInt();
            list.remove(del - 1);
            break;
        case 3:
            System.out.print("Enter line to append: ");
            list.add(sc.nextLine());
            break;
        case 4:
            System.out.print("Enter line number to modify: ");
            int mod = sc.nextInt();
            sc.nextLine();
            System.out.print("Enter new line: ");
            list.set(mod - 1, sc.nextLine());
            break;
        case 5:
            // Write back to file
            BufferedWriter bw = new BufferedWriter(new FileWriter(filename));
            for (String s : list) {
                bw.write(s);
                bw.newLine();
            }
```

```
                    bw.close();
                    System.out.println("File saved. Exiting...");
                    break;
                default:
                    System.out.println("Invalid choice!");
            }
        } while (choice != 5);
        sc.close();
    }
}
```

Assignment 2

Q11: Program to define a thread for printing text on output screen for ‘n’ number of times. Create 3 threads and run them. Pass the text ‘n’ parameters to the thread constructor. Example:

i. First thread prints “COVID19” 10 times.

ii. Second thread prints “LOCKDOWN2020” 20 times

iii. Third thread prints “VACCINATED2021” 30 times

```
class PrintThread extends Thread {
    String message;
    int times;
    // Constructor
    PrintThread(String message, int times) {
        this.message = message;
        this.times = times;
    }
    // Thread task
    public void run() {
        for (int i = 1; i <= times; i++) {
            System.out.println(message);
        }
    }
}
public class setA1 {
    public static void main(String[] args) {
        // Creating threads
        PrintThread t1 = new PrintThread("COVID19", 10);
        PrintThread t2 = new PrintThread("LOCKDOWN2020", 20);
        PrintThread t3 = new PrintThread("VACCINATED2021", 30);
        // Starting threads
        t1.start();
        t2.start();
        t3.start();
```

```
    }
}
```

Q12: Write a program in which thread sleep for 6 sec in the loop in reverse order from 100 to 1 and change the name of thread.

```
class ReverseThread extends Thread {
    public void run() {
        try {
            for (int i = 10; i >= 1; i--) {
                System.out.println(Thread.currentThread().getName() + " : " + i);
                Thread.sleep(6000); // sleep for 6 seconds
            }
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}
public class setA2 {
    public static void main(String[] args) {
        ReverseThread t = new ReverseThread();
        // Change thread name
        t.setName("ReverseThread");
        // Start thread
        t.start();
    }
}
```

Q13: Write a program to solve producer consumer problem in which a producer produces a value and consumer consume the value before producer generate the next value. (Hint: use thread synchronization)

```
class SharedResource {
    int value;
    boolean available = false;
    // Producer method
    synchronized void produce(int v) {
        try {
            while (available) {
                wait();
            }
            value = v;
            System.out.println("Produced: " + value);
            available = true;
            notify();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
```

```
    }
    // Consumer method
    synchronized void consume() {
        try {
            while (!available) {
                wait();
            }
            System.out.println("Consumed: " + value);
            available = false;
            notify();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}
// Producer Thread
class Producer extends Thread {
    SharedResource sr;
    Producer(SharedResource sr) {
        this.sr = sr;
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            sr.produce(i);
        }
    }
}
// Consumer Thread
class Consumer extends Thread {
    SharedResource sr;
    Consumer(SharedResource sr) {
        this.sr = sr;
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            sr.consume();
        }
    }
}
public class setA3 {
    public static void main(String[] args) {
        SharedResource sr = new SharedResource();
        Producer p = new Producer(sr);
        Consumer c = new Consumer(sr);
        p.start();
```

```
        c.start();
    }
}
```

Q14: Write a program to calculate the sum and average of an array of 1000 integers (generated randomly) using 10 threads. Each thread calculates the sum of 100 integers. Use these values to calculate average. [Use join method].

```
import java.util.Random;
class SumThread extends Thread {
    int[] arr;
    int start, end;
    int partialSum = 0;
    SumThread(int[] arr, int start, int end) {
        this.arr = arr;
        this.start = start;
        this.end = end;
    }
    public void run() {
        for (int i = start; i < end; i++) {
            partialSum += arr[i];
        }
        System.out.println(
            "Thread " + getName() + " sum = " + partialSum
        );
    }
}
public class setB1 {
    public static void main(String[] args) throws InterruptedException {
        int[] arr = new int[1000];
        Random rand = new Random();
        // Generate random numbers
        for (int i = 0; i < 1000; i++) {
            arr[i] = rand.nextInt(100); // 0 to 99
        }
        SumThread[] threads = new SumThread[10];
        // Create and start threads
        for (int i = 0; i < 10; i++) {
            threads[i] = new SumThread(arr, i * 100, (i + 1) * 100);
            threads[i].start();
        }
        int totalSum = 0;
        // Join threads and collect results
        for (int i = 0; i < 10; i++) {
            threads[i].join();
```

```
            totalSum += threads[i].partialSum;
        }
        double average = totalSum / 1000.0;
        System.out.println("\nTotal Sum = " + totalSum);
        System.out.println("Average = " + average);
    }
}
```

Q15: Write a program for a simple search engine. Accept a string to be searched. Search for the string in all text files in the current folder. Use a separate thread for each file. The result should display the filename, line number where the string is found.

```
import java.io.*;
class SearchThread extends Thread {
    File file;
    String word;
    SearchThread(File file, String word) {
        this.file = file;
        this.word = word;
    }
    public void run() {
        try {
            BufferedReader br = new BufferedReader(new FileReader(file));
            String line;
            int lineNo = 1;
            while ((line = br.readLine()) != null) {
                if (line.contains(word)) {
                    System.out.println(
                        "Found in file: " + file.getName() +
                        " | Line: " + lineNo +
                        " | Text: " + line
                    );
                }
                lineNo++;
            }
            br.close();
        } catch (Exception e) {
            System.out.println("Error reading file " + file.getName());
        }
    }
}
public class setB2 {
    public static void main(String[] args) throws IOException {
        BufferedReader input =
            new BufferedReader(new InputStreamReader(System.in));
```

```
        System.out.print("Enter word to search: ");
        String word = input.readLine();
        File folder = new File(".");
        File[] files = folder.listFiles((dir, name) -> name.endsWith(".txt"));
        if (files == null || files.length == 0) {
            System.out.println("No text files found.");
            return;
        }
        // Create a thread for each file
        for (File file : files) {
            SearchThread t = new SearchThread(file, word);
            t.start();
        }
    }
}
```

Q16: Write a program that implements a multi-thread application that has three threads. First thread generates random integer every 1 second and if the value is even, second thread computes the square of the number and prints. If the value is odd, the third thread will print the value of cube of the number.

```
import java.util.Random;
class NumberGenerator extends Thread {
    int number;
    public void run() {
        Random rand = new Random();
        try {
            while (true) {
                number = rand.nextInt(10) + 1; // 1 to 10
                System.out.println("\nGenerated Number: " + number);
                if (number % 2 == 0) {
                    new SquareThread(number).start();
                } else {
                    new CubeThread(number).start();
                }
                Thread.sleep(1000); // 1 second
            }
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted");
        }
    }
}
class SquareThread extends Thread {
    int num;
    SquareThread(int num) {
        this.num = num;
```

```
    }
    public void run() {
        System.out.println("Square: " + (num * num));
    }
}
class CubeThread extends Thread {
    int num;
    CubeThread(int num) {
        this.num = num;
    }
    public void run() {
        System.out.println("Cube: " + (num * num * num));
    }
}
public class setB3 {
    public static void main(String[] args) {
        NumberGenerator t = new NumberGenerator();
        t.start();
    }
}
```

Q17: Write a program that simulates a traffic light. The program lets the user select one of three lights: red, yellow, or green with radio buttons. On selecting a button, an appropriate message with “stop” or “ready” or “go”should appear above the buttons in a selected color. Initially there is no message shown.

```
import java.awt.*;
import java.awt.event.*;
public class setC1 extends Frame implements ItemListener {
    Checkbox red, yellow, green;
    Label message;
    setC1() {
        // Set layout
        setLayout(new FlowLayout());
        // Create CheckboxGroup
        CheckboxGroup cbg = new CheckboxGroup();
        // Create radio buttons
        red = new Checkbox("Red", cbg, false);
        yellow = new Checkbox("Yellow", cbg, false);
        green = new Checkbox("Green", cbg, false);
        // Create label
        message = new Label("");
        // Add components
        add(red);
        add(yellow);
        add(green);
```

```
        add(message);
        // Register listeners
        red.addItemListener(this);
        yellow.addItemListener(this);
        green.addItemListener(this);
        // Frame settings
        setTitle("Traffic Light Simulation");
        setSize(300, 150);
        setVisible(true);
        // Close window
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                dispose();
            }
        });
    }
    // Handle radio button selection
    public void itemStateChanged(ItemEvent ie) {
        if (red.getState())
            message.setText("STOP");
        else if (yellow.getState())
            message.setText("READY");
        else if (green.getState())
            message.setText("GO");
    }
    public static void main(String[] args) {
        new setC1();
    }
}
```

Q18: Write a program to create a thread for moving a ball inside a panel vertically. The ball should be created when the user clicks on the start button.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class BallPanel extends JPanel implements Runnable {
    int y = 10;          // vertical position of ball
    Thread t;
    BallPanel() {
        setBackground(Color.WHITE);
    }
    // Start thread
    public void startBall() {
        t = new Thread(this);
```

```
        t.start();
    }
    // Thread logic
    public void run() {
        while (true) {
            y += 5;      // move down
            if (y > getHeight() - 30)
                y = 10;  // reset to top

            repaint();
            try {
                Thread.sleep(100);
            } catch (Exception e) {
            }
        }
    }
    // Draw ball
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.RED);
        g.fillOval(100, y, 30, 30);
    }
}
public class setC2 extends JFrame implements ActionListener {
    BallPanel panel;
    JButton startBtn;
    setC2() {
        panel = new BallPanel();
        startBtn = new JButton("Start");
        startBtn.addActionListener(this);
        add(panel, BorderLayout.CENTER);
        add(startBtn, BorderLayout.SOUTH);
        setTitle("Moving Ball Using Thread");
        setSize(300, 300);
        setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    // Button click event
    public void actionPerformed(ActionEvent ae) {
        panel.startBall();
    }
    public static void main(String[] args) {
        new setC2();
    }
}
```

Q19: Using the concepts of thread synchronization create two threads as sender and receiver. Sender thread will set a message to the receiver thread that will display the message on console. The sender thread accepts the input message from console. Continue this process until sender sets the message as “Good Bye Corona”.

```
import java.util.Scanner;
// Shared class
class Message {
    String msg;
    boolean hasMessage = false;
    // Sender sends message
    synchronized void send(String msg) {
        while (hasMessage) {
            try {
                wait();
            } catch (Exception e) {}
        }
        this.msg = msg;
        hasMessage = true;
        notify();
    }
    // Receiver receives message
    synchronized String receive() {
        while (!hasMessage) {
            try {
                wait();
            } catch (Exception e) {}
        }
        hasMessage = false;
        notify();
        return msg;
    }
}
// Sender Thread
class Sender extends Thread {
    Message m;
    Scanner sc = new Scanner(System.in);
    Sender(Message m) {
        this.m = m;
    }
    public void run() {
        String text;
        do {
            System.out.print("Enter message: ");
            text = sc.nextLine();
```

```
            m.send(text);
        } while (!text.equals("Good Bye Corona"));
    }
}
// Receiver Thread
class Receiver extends Thread {
    Message m;
    Receiver(Message m) {
        this.m = m;
    }
    public void run() {
        String text;
        do {
            text = m.receive();
            System.out.println("Received message: " + text);
        } while (!text.equals("Good Bye Corona"));
    }
}
// Main class
public class setC3 {
    public static void main(String[] args) {
        Message m = new Message();
        Sender s = new Sender(m);
        Receiver r = new Receiver(m);
        s.start();
        r.start();
    }
}
```

Assignment 3

Q20: Create a PROJECT table with fields project_id, Project_name, Project_description, Project_Status. etc. Insert values in the table. Display all the details of the PROJECT table in a tabular format on the screen.(using swing).

```
import java.sql.*;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
public class setA1 {
    public static void main(String[] args) {
        try {
            // Load JDBC Driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Establish Connection
            Connection con = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/testdb",
```

```
                "root",
                ""
            );
            Statement stmt = con.createStatement();
            // Create PROJECT table
            String createTable = "CREATE TABLE IF NOT EXISTS PROJECT ("
                    + "project_id INT , "
                    + "project_name VARCHAR(50), "
                    + "project_description VARCHAR(100), "
                    + "project_status VARCHAR(20))";
            stmt.executeUpdate(createTable);
            // Insert values
            stmt.executeUpdate("INSERT INTO PROJECT VALUES (1,'Library System','College
Library Project','Completed')");
            stmt.executeUpdate("INSERT INTO PROJECT VALUES (2,'Payroll System','Employee
Salary Project','Running')");
            stmt.executeUpdate("INSERT INTO PROJECT VALUES (3,'Online Exam','MCQ Exam
System','Pending')");
            // Fetch data
            ResultSet rs = stmt.executeQuery("SELECT * FROM PROJECT");
            // Create Swing Frame
            JFrame frame = new JFrame("PROJECT Table");
            frame.setSize(600, 300);
            // Table Model
            DefaultTableModel model = new DefaultTableModel();
            JTable table = new JTable(model);
            // Add columns
            model.addColumn("Project ID");
            model.addColumn("Project Name");
            model.addColumn("Project Description");
            model.addColumn("Project Status");
            // Add rows
            while (rs.next()) {
                model.addRow(new Object[]{
                        rs.getInt(1),
                        rs.getString(2),
                        rs.getString(3),
                        rs.getString(4)
                });
            }
            // Add table to frame
            frame.add(new JScrollPane(table));
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setVisible(true);
            con.close();
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Q21: Write a program to display information about the database and list all the tables in the database. (Use DatabaseMetaData).

```
import java.sql.*;
public class setA2 {
    public static void main(String[] args) {
        try {
            // Load Driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Create Connection
            Connection con = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/testdb",
                    "root",
                    ""
            );
            // Get DatabaseMetaData object
            DatabaseMetaData dbmd = con.getMetaData();
            // Display database information
            System.out.println("Database Product Name : " + dbmd.getDatabaseProductName());
            System.out.println("Database Product Version : " +
dbmd.getDatabaseProductVersion());
            System.out.println("Driver Name : " + dbmd.getDriverName());
            System.out.println("Driver Version : " + dbmd.getDriverVersion());
            System.out.println("User Name : " + dbmd.getUserName());
            System.out.println("\nList of Tables in Database:\n");
            // Get all tables
            ResultSet rs = dbmd.getTables(null, null, "%", new String[]{"TABLE"});
            while (rs.next()) {
                System.out.println(rs.getString("TABLE_NAME"));
            }
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Q22: Write a program to display information about all columns in the DONAR table using ResultSetMetaData.

```
import java.sql.*;
```

```
public class setA3 {
    public static void main(String[] args) {
        try {
            // Load JDBC Driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Establish Connection
            Connection con = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/testdb",
                    "root",
                    ""
            );
            // Create Statement
            Statement stmt = con.createStatement();
            // Execute query
            ResultSet rs = stmt.executeQuery("SELECT * FROM DONAR");
            // Get ResultSetMetaData
            ResultSetMetaData rsmd = rs.getMetaData();
            int columnCount = rsmd.getColumnCount();
            System.out.println("Total Columns: " + columnCount);
            System.out.println("----------------------------------------");
            // Display column information
            for (int i = 1; i <= columnCount; i++) {
                System.out.println("Column Number : " + i);
                System.out.println("Column Name   : " + rsmd.getColumnName(i));
                System.out.println("Column Type   : " + rsmd.getColumnTypeName(i));
                System.out.println("Column Size   : " + rsmd.getColumnDisplaySize(i));
                System.out.println("Nullable      : " + rsmd.isNullable(i));
                System.out.println("----------------------------------------");
            }
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Q23: Create a MOBILE table with fields Model_Number, Model_Name, Model_Color, Sim_Type, NetworkType, BatteryCapacity, InternalStorage, RAM and ProcessorType. Insert values in the table. Write a menu driven program to pass the input using Command line argument to perform the following operations on MOBILE table. 1. Insert 2. Modify 3. Delete 4. Search 5. View All 6. Exit

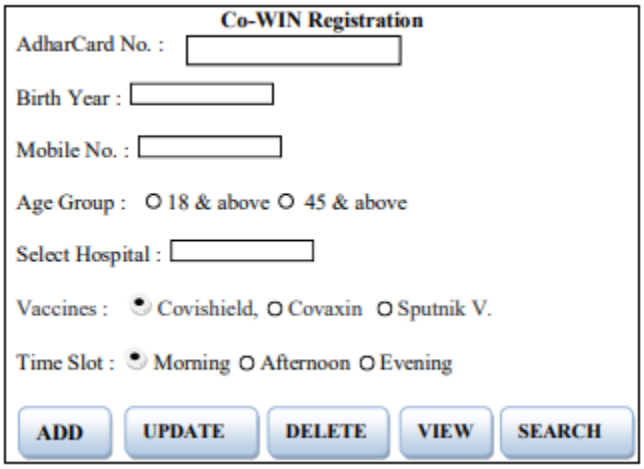
```
import java.sql.*;
import java.util.Scanner;
public class setB1 {
    static Connection con;
```

```
static Scanner sc = new Scanner(System.in);
public static void main(String[] args) {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/test", "root", "");
        int ch;
        do {
            System.out.println("\n--- MOBILE TABLE MENU ---");
            System.out.println("1. Insert");
            System.out.println("2. Modify");
            System.out.println("3. Delete");
            System.out.println("4. Search");
            System.out.println("5. View All");
            System.out.println("6. Exit");
            System.out.print("Enter choice: ");
            ch = sc.nextInt();
            switch (ch) {
                case 1:
                    insert();
                    break;
                case 2:
                    modify();
                    break;
                case 3:
                    delete();
                    break;
                case 4:
                    search();
                    break;
                case 5:
                    view();
                    break;
                case 6:
                    System.out.println("Exiting...");
                    break;
                default:
                    System.out.println("Invalid Choice");
            }
        } while (ch != 6);
        con.close();
    } catch (Exception e) {
        System.out.println(e);
    }
}
```

```
static void insert() throws SQLException {
    String sql = "INSERT INTO mobile VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)";
    PreparedStatement ps = con.prepareStatement(sql);
    System.out.print("Model Number: ");
    ps.setInt(1, sc.nextInt());
    sc.nextLine();
    System.out.print("Model Name: ");
    ps.setString(2, sc.nextLine());
    System.out.print("Model Color: ");
    ps.setString(3, sc.nextLine());
    System.out.print("SIM Type: ");
    ps.setString(4, sc.nextLine());
    System.out.print("Network Type: ");
    ps.setString(5, sc.nextLine());
    System.out.print("Battery Capacity: ");
    ps.setInt(6, sc.nextInt());
    System.out.print("Internal Storage: ");
    ps.setInt(7, sc.nextInt());
    System.out.print("RAM: ");
    ps.setInt(8, sc.nextInt());
    sc.nextLine();
    System.out.print("Processor Type: ");
    ps.setString(9, sc.nextLine());
    ps.executeUpdate();
    System.out.println("Record Inserted Successfully");
}
static void modify() throws SQLException {
    String sql = "UPDATE MOBILE SET Model_Color=? WHERE Model_Number=?";
    PreparedStatement ps = con.prepareStatement(sql);
    System.out.print("Enter Model Number: ");
    ps.setInt(2, sc.nextInt());
    sc.nextLine();
    System.out.print("New Color: ");
    ps.setString(1, sc.nextLine());
    ps.executeUpdate();
    System.out.println("Record Updated Successfully");
}
static void delete() throws SQLException {
    String sql = "DELETE FROM MOBILE WHERE Model_Number=?";
    PreparedStatement ps = con.prepareStatement(sql);
    System.out.print("Enter Model Number: ");
    ps.setInt(1, sc.nextInt());
    ps.executeUpdate();
    System.out.println("Record Deleted Successfully");
}
```

```
    static void search() throws SQLException {
        String sql = "SELECT * FROM MOBILE WHERE Model_Number=?";
        PreparedStatement ps = con.prepareStatement(sql);
        System.out.print("Enter Model Number: ");
        ps.setInt(1, sc.nextInt());
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            System.out.println(
                rs.getInt(1) + " " +
                rs.getString(2) + " " +
                rs.getString(3) + " " +
                rs.getString(4) + " " +
                rs.getString(5) + " " +
                rs.getInt(6) + " " +
                rs.getInt(7) + " " +
                rs.getInt(8) + " " +
                rs.getString(9)
            );
        } else {
            System.out.println("Record Not Found");
        }
    }
    static void view() throws SQLException {
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("SELECT * FROM MOBILE");
        while (rs.next()) {
            System.out.println(
                rs.getInt(1) + " " +
                rs.getString(2) + " " +
                rs.getString(3) + " " +
                rs.getString(4) + " " +
                rs.getString(5) + " " +
                rs.getInt(6) + " " +
                rs.getInt(7) + " " +
                rs.getInt(8) + " " +
                rs.getString(9)
            );
        }
    }
}
```

Q24: Design a following Registration form and raise an appropriate exception if invalid information is entered like Birth Year ‘0000’




```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
// Custom Exception
class InvalidBirthYearException extends Exception {
    public InvalidBirthYearException(String msg) {
        super(msg);
    }
}
public class setB2 extends JFrame implements ActionListener {
    JTextField txtAadhar, txtBirthYear, txtMobile, txtHospital;
    JRadioButton rb18, rb45, rbCovishield, rbCovaxin, rbSputnik;
    JRadioButton rbMorning, rbAfternoon, rbEvening;
    JButton btnAdd;
    public setB2() {
        setTitle("Co-WIN Registration");
        setSize(450, 420);
        setLayout(null);
        JLabel lblTitle = new JLabel("Co-WIN Registration");
        lblTitle.setFont(new Font("Arial", Font.BOLD, 16));
        lblTitle.setBounds(140, 10, 200, 30);
        add(lblTitle);
        JLabel lblAadhar = new JLabel("Aadhar Card No:");
        lblAadhar.setBounds(30, 60, 120, 25);
        add(lblAadhar);
        txtAadhar = new JTextField();
        txtAadhar.setBounds(160, 60, 200, 25);
        add(txtAadhar);
        JLabel lblBirth = new JLabel("Birth Year:");
        lblBirth.setBounds(30, 100, 120, 25);
```

```
add(lblBirth);
txtBirthYear = new JTextField();
txtBirthYear.setBounds(160, 100, 200, 25);
add(txtBirthYear);
JLabel lblMobile = new JLabel("Mobile No:");
lblMobile.setBounds(30, 140, 120, 25);
add(lblMobile);
txtMobile = new JTextField();
txtMobile.setBounds(160, 140, 200, 25);
add(txtMobile);
JLabel lblAge = new JLabel("Age Group:");
lblAge.setBounds(30, 180, 120, 25);
add(lblAge);
rb18 = new JRadioButton("18 & above");
rb45 = new JRadioButton("45 & above");
rb18.setBounds(160, 180, 100, 25);
rb45.setBounds(260, 180, 120, 25);
ButtonGroup bgAge = new ButtonGroup();
bgAge.add(rb18);
bgAge.add(rb45);
add(rb18);
add(rb45);
JLabel lblHospital = new JLabel("Select Hospital:");
lblHospital.setBounds(30, 220, 120, 25);
add(lblHospital);
txtHospital = new JTextField();
txtHospital.setBounds(160, 220, 200, 25);
add(txtHospital);
JLabel lblVaccine = new JLabel("Vaccines:");
lblVaccine.setBounds(30, 260, 120, 25);
add(lblVaccine);
rbCovishield = new JRadioButton("Covishield");
rbCovaxin = new JRadioButton("Covaxin");
rbSputnik = new JRadioButton("Sputnik V");
rbCovishield.setBounds(160, 260, 100, 25);
rbCovaxin.setBounds(260, 260, 100, 25);
rbSputnik.setBounds(160, 290, 100, 25);
ButtonGroup bgVaccine = new ButtonGroup();
bgVaccine.add(rbCovishield);
bgVaccine.add(rbCovaxin);
bgVaccine.add(rbSputnik);
add(rbCovishield);
add(rbCovaxin);
add(rbSputnik);
JLabel lblSlot = new JLabel("Time Slot:");
```

```
        lblSlot.setBounds(30, 320, 120, 25);
        add(lblSlot);
        rbMorning = new JRadioButton("Morning");
        rbAfternoon = new JRadioButton("Afternoon");
        rbEvening = new JRadioButton("Evening");
        rbMorning.setBounds(160, 320, 90, 25);
        rbAfternoon.setBounds(250, 320, 100, 25);
        rbEvening.setBounds(160, 350, 100, 25);
        ButtonGroup bgSlot = new ButtonGroup();
        bgSlot.add(rbMorning);
        bgSlot.add(rbAfternoon);
        bgSlot.add(rbEvening);
        add(rbMorning);
        add(rbAfternoon);
        add(rbEvening);
        btnAdd = new JButton("ADD");
        btnAdd.setBounds(170, 380, 80, 30);
        add(btnAdd);
        btnAdd.addActionListener(this);
        setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public void actionPerformed(ActionEvent e) {
        try {
            int birthYear = Integer.parseInt(txtBirthYear.getText());
            if (birthYear == 0) {
                throw new InvalidBirthYearException("Invalid Birth Year: 0000 not allowed");
            }
            JOptionPane.showMessageDialog(this,
                    "Registration Successful!",
                    "Success",
                    JOptionPane.INFORMATION_MESSAGE);
        } catch (InvalidBirthYearException ex) {
            JOptionPane.showMessageDialog(this,
                    ex.getMessage(),
                    "Error",
                    JOptionPane.ERROR_MESSAGE);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this,
                    "Please enter valid numeric Birth Year",
                    "Error",
                    JOptionPane.ERROR_MESSAGE);
        }
    }
    public static void main(String[] args) {
```

```
        new setB2();
    }
}
```

Q25: Create tables : Course (courseid, coursename, courseinstructor) and Student (studentid, studentname, studentclass). Course and Student have a many to many relationship. Create a GUI based system for performing the following operations on the tables:

Course : Add Course, View All students of a specific course

Student : Add Student, Delete Student, View All students, Search student.

```
import java.util.*;
class Student {
    int studentId;
    String studentName;
    String studentClass;
    Student(int id, String name, String sclass) {
        studentId = id;
        studentName = name;
        studentClass = sclass;
    }
    public String toString() {
        return studentId + "\t" + studentName + "\t" + studentClass;
    }
}
class Course {
    int courseId;
    String courseName;
    String instructor;
    List<Student> students = new ArrayList<>();
    Course(int id, String name, String instr) {
        courseId = id;
        courseName = name;
        instructor = instr;
    }
}
public class setC1 {
    static Scanner sc = new Scanner(System.in);
    static List<Student> studentList = new ArrayList<>();
    static List<Course> courseList = new ArrayList<>();
    public static void main(String[] args) {
        int choice;
        do {
            System.out.println("\n--- MENU ---");
            System.out.println("1. Add Student");
```

```
        System.out.println("2. Delete Student");
        System.out.println("3. View All Students");
        System.out.println("4. Search Student");
        System.out.println("5. Add Course");
        System.out.println("6. View Students of a Course");
        System.out.println("7. Exit");
        System.out.print("Enter choice: ");
        choice = sc.nextInt();
        switch (choice) {
            case 1: addStudent(); break;
            case 2: deleteStudent(); break;
            case 3: viewStudents(); break;
            case 4: searchStudent(); break;
            case 5: addCourse(); break;
            case 6: viewCourseStudents(); break;
            case 7: System.out.println("Exiting..."); break;
            default: System.out.println("Invalid choice");
        }
    } while (choice != 7);
}
static void addStudent() {
    System.out.print("Enter Student ID: ");
    int id = sc.nextInt();
    sc.nextLine();
    System.out.print("Enter Student Name: ");
    String name = sc.nextLine();
    System.out.print("Enter Class: ");
    String cls = sc.nextLine();
    studentList.add(new Student(id, name, cls));
    System.out.println("Student added successfully.");
}
static void deleteStudent() {
    System.out.print("Enter Student ID to delete: ");
    int id = sc.nextInt();
    studentList.removeIf(s -> s.studentId == id);
    System.out.println("Student deleted if existed.");
}
static void viewStudents() {
    System.out.println("\nID\tName\tClass");
    for (Student s : studentList) {
        System.out.println(s);
    }
}
static void searchStudent() {
    System.out.print("Enter Student ID to search: ");
```

```
        int id = sc.nextInt();
        for (Student s : studentList) {
            if (s.studentId == id) {
                System.out.println("Found: " + s);
                return;
            }
        }
        System.out.println("Student not found.");
    }
    static void addCourse() {
        System.out.print("Enter Course ID: ");
        int id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Course Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Instructor Name: ");
        String instr = sc.nextLine();
        Course c = new Course(id, name, instr);
        System.out.print("How many students to enroll? ");
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            System.out.print("Enter Student ID: ");
            int sid = sc.nextInt();
            for (Student s : studentList) {
                if (s.studentId == sid) {
                    c.students.add(s);
                }
            }
        }
        courseList.add(c);
        System.out.println("Course added successfully.");
    }
    static void viewCourseStudents() {
        System.out.print("Enter Course ID: ");
        int id = sc.nextInt();
        for (Course c : courseList) {
            if (c.courseId == id) {
                System.out.println("\nStudents in " + c.courseName + ":");
                for (Student s : c.students) {
                    System.out.println(s);
                }
                return;
            }
        }
        System.out.println("Course not found.");
```

```
    }
}
```

Q26: Create the following tables and relations, for an INVESTMENT firm EMP(empid ,empname, empaddress, empcontact, empage) INVESTOR(invno, invname , invdate, invamt) An employee may invest in one or more investments, hence he can be an investor. But an investor need not be an employee of the firm. Insert sufficient number of records in the relations / tables with appropriate values.

i. Display the List the distinct names of person who are either employees, or investors or both.

ii. List the names of employees who are not investors

```
import java.util.*;
class Employee {
    int empId;
    String empName;
    Employee(int id, String name) {
        empId = id;
        empName = name;
    }
}
class Investor {
    int invId;
    String invName;
    Investor(int id, String name) {
        invId = id;
        invName = name;
    }
}
public class setC2 {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>();
        List<Investor> investors = new ArrayList<>();
        // Adding Employees
        employees.add(new Employee(1, "Amit"));
        employees.add(new Employee(2, "Neha"));
        employees.add(new Employee(3, "Rohit"));
        // Adding Investors
        investors.add(new Investor(101, "Neha"));   // Employee + Investor
        investors.add(new Investor(102, "Suresh")); // Only Investor
        // i) Distinct names of employees or investors or both
        Set<String> allPersons = new HashSet<>();
        for (Employee e : employees)
            allPersons.add(e.empName);
        for (Investor i : investors)
            allPersons.add(i.invName);
```

```
        System.out.println("Distinct Employees or Investors:");
        for (String name : allPersons)
            System.out.println(name);
        // ii) Employees who are NOT investors
        Set<String> investorNames = new HashSet<>();
        for (Investor i : investors)
            investorNames.add(i.invName);
        System.out.println("\nEmployees who are not Investors:");
        for (Employee e : employees) {
            if (!investorNames.contains(e.empName)) {
                System.out.println(e.empName);
            }
        }
    }
}
```

Assignment 4

1.C:\Java-II\apache-tomcat-9.0.115\webapps\RequestInfo\WEB-INF\classes =>cmd javac –cp ".;C:\ Java-II\apache-tomcat-9.0.115\lib\servlet-api.jar" RequestInfoServlet.java

2.C:\Java-II\apache-tomcat-9.0.115\bin =>startup.bat

3.start xampp =>apache =>start

4.chrome =>http://localhost:8080/RequestInfo/info

Q27: Design a servlet that provides information about a HTTP request from a client, such as IP address and browser type. The servlet also provides information about the server on which the servlet is running, such as the operating system type, and the names of currently loaded servlets.

(C:\Java-II\apache-tomcat-9.0.115\webapps\RequestInfo\WEB-INF\classes\`RequestInfoServlet.java`)

```
import java.io.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class RequestInfoServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h2>HTTP Request and Server Information</h2>");
        // CLIENT INFO
        out.println("<h3>Client Information</h3>");
        out.println("IP Address: " + request.getRemoteAddr() + "<br>");
        out.println("Browser Type: " + request.getHeader("User-Agent") + "<br>");
```

```
        // SERVER INFO
        out.println("<h3>Server Information</h3>");
        out.println("Server Info: " + getServletContext().getServerInfo() + "<br>");
        out.println("Operating System: " + System.getProperty("os.name") + "<br>");
        // Loaded Servlets
        out.println("<h3>Loaded Servlets</h3>");
        ServletContext context = getServletContext();
        Set<String> servlets = context.getServletRegistrations().keySet();
        for (String name : servlets) {
            out.println(name + "<br>");
        }
        out.println("</body></html>");
    }
}
```

(C:\Java-II\apache-tomcat-9.0.115\webapps\RequestInfo\WEB-INF\web.xml)

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         version="3.1">

    <servlet>
        <servlet-name>RequestInfoServlet</servlet-name>
        <servlet-class>RequestInfoServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>RequestInfoServlet</servlet-name>
        <url-pattern>/info</url-pattern>
    </servlet-mapping>

</web-app>
```

Q28: Write a Program to make use of following JSP implicit objects:

i. out: To display current Date and Time.

ii. request: To get header information.

iii. response: To Add Cookie

iv. config: get the parameters value defined in

v. application: get the parameter value defined in

vi. session: Display Current Session ID

vii. pageContext: To set and get the attributes.

viii. page: get the name of Generated Servlet

(C:\Java-II\apache-tomcat-9.0.115\webapps\ImplicitDemo\implicit.jsp)

```
<%@ page import="java.util.Date" %>
<html>
<head>
    <title>JSP Implicit Objects Demo</title>
</head>
<body>
<h2>JSP Implicit Objects Demo</h2>
<!-- i. out object -->
<h3>1. out (Current Date and Time)</h3>
<%
    out.println("Current Date & Time: " + new Date());
%>
<!-- ii. request object -->
<h3>2. request (Header Information)</h3>
<%
    String browser = request.getHeader("User-Agent");
    out.println("Browser Info: " + browser + "<br>");
    out.println("Client IP: " + request.getRemoteAddr());
%>
<!-- iii. response object -->
<h3>3. response (Add Cookie)</h3>
<%
    Cookie ck = new Cookie("username", "Nilesh");
    response.addCookie(ck);
    out.println("Cookie Added Successfully!");
%>
<!-- iv. config object -->
<h3>4. config (init-param value)</h3>
<%
    String driver = config.getInitParameter("driver");
    out.println("Driver Name: " + driver);
%>
<!-- v. application object -->
<h3>5. application (context-param value)</h3>
<%
    String appName = application.getInitParameter("AppName");
    out.println("Application Name: " + appName);
%>
<!-- vi. session object -->
<h3>6. session (Session ID)</h3>
<%
    out.println("Session ID: " + session.getId());
```

```
%>
<!-- vii. pageContext object -->
<h3>7. pageContext (Set and Get Attribute)</h3>
<%
    pageContext.setAttribute("msg", "Welcome to JSP");
    String message = (String) pageContext.getAttribute("msg");
    out.println("Message: " + message);
%>
<!-- viii. page object -->
<h3>8. page (Generated Servlet Name)</h3>
<%
    out.println("Generated Servlet Name: " + page.getClass().getName());
%>
</body>
</html>
```

(C:\Java-II\apache-tomcat-9.0.115\webapps\ImplicitDemo\WEB-INF\web.xml)

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         version="3.1">
    <context-param>
        <param-name>AppName</param-name>
        <param-value>Implicit Object Demo Application</param-value>
    </context-param>
    <jsp-config>
        <jsp-property-group>
            <url-pattern>*.jsp</url-pattern>
            <init-param>
                <param-name>driver</param-name>
                <param-value>com.mysql.jdbc.Driver</param-value>
            </init-param>
        </jsp-property-group>
    </jsp-config>
</web-app>
```

Q29: Write a program to create a Online Book purchase. User must be login and then purchase the book. Each page should have a page total. The last page should display a total book and bill, which consists of a page total of what ever the purchase has been done and print the total. (Use HttpSession)

(C:\Java-II\apache-tomcat-9.0.115\webapps\OnlineBook\login.html)

```
<form action="login" method="post">
Username: <input type="text" name="username"><br>
Password: <input type="password" name="password"><br>
<input type="submit" value="Login">
</form>
```

(C:\Java-II\apache-tomcat-9.0.115\webapps\OnlineBook\WEB-INF\web.xml)

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         version="3.1">
    <servlet>
        <servlet-name>LoginServlet</servlet-name>
        <servlet-class>LoginServlet</servlet-class>
    </servlet>
    <servlet>
        <servlet-name>BookServlet</servlet-name>
        <servlet-class>BookServlet</servlet-class>
    </servlet>
    <servlet>
        <servlet-name>BillServlet</servlet-name>
        <servlet-class>BillServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>LoginServlet</servlet-name>
        <url-pattern>/login</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>BookServlet</servlet-name>
        <url-pattern>/books</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>BillServlet</servlet-name>
        <url-pattern>/bill</url-pattern>
    </servlet-mapping>
</web-app>
```

(C:\Java-II\apache-tomcat-9.0.115\webapps\OnlineBook\WEB-INF\classes\LoginServlet.java)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String user = request.getParameter("username");
        String pass = request.getParameter("password");
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        if (user.equals("admin") && pass.equals("1234")) {
            HttpSession session = request.getSession();
            session.setAttribute("user", user);
            session.setAttribute("total", 0.0);
```

```
            session.setAttribute("count", 0);
            response.sendRedirect("books");
        } else {
            out.println("<h3>Invalid Login!</h3>");
        }
    }
}
```

(C:\Java-II\apache-tomcat-9.0.115\webapps\OnlineBook\WEB-INF\classes\BookServlet.java)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class BookServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h2>Online Book Store</h2>");
        out.println("<form method='post'>");
        out.println("<input type='checkbox' name='book' value='500'> Java (500)<br>");
        out.println("<input type='checkbox' name='book' value='600'> Python (600)<br>");
        out.println("<input type='checkbox' name='book' value='700'> Spring (700)<br><br>");
        out.println("<input type='submit' value='Purchase'>");
        out.println("</form>");
        out.println("<br><a href='bill'>Go To Final Bill</a>");
    }
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession();
        String[] books = request.getParameterValues("book");
        double pageTotal = 0;
        if (books != null) {
            for (String price : books) {
                pageTotal += Double.parseDouble(price);
            }
        }
        double total = (double) session.getAttribute("total");
        int count = (int) session.getAttribute("count");
        total += pageTotal;
        count += (books == null ? 0 : books.length);
        session.setAttribute("total", total);
        session.setAttribute("count", count);
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
```

```
        out.println("<h3>Page Total: " + pageTotal + "</h3>");
        out.println("<a href='books'>Buy More</a><br>");
        out.println("<a href='bill'>Go To Final Bill</a>");
    }
}
```

(C:\Java-II\apache-tomcat-9.0.115\webapps\OnlineBook\WEB-INF\classes\BillServlet.java)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class BillServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        if (session != null) {
            double total = (double) session.getAttribute("total");
            int count = (int) session.getAttribute("count");
            out.println("<h2>Final Bill</h2>");
            out.println("Total Books Purchased: " + count + "<br>");
            out.println("Total Amount: " + total + "<br>");
            session.invalidate();
        } else {
            out.println("Session Expired!");
        }
    }
}
```

Q30: Design an HTML page which passes customer number to a search servlet. The servlet searches for the customer number in a database (customer table) and returns customer details if found the number otherwise display error message.

(C:\Java-II\apache-tomcat-9.0.115\webapps\CustomerSearch\search.html)

```
<html>
<body>
<h2>Customer Search</h2>
<form action="search" method="post">
    Enter Customer Number:
    <input type="text" name="custno">
    <input type="submit" value="Search">
</form>
</body>
</html>
```

(C:\Java-II\apache-tomcat-9.0.115\webapps\CustomerSearch\WEB-INF\classes\SearchServlet.java)

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class SearchServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String custno = request.getParameter("custno");
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/customerdb",
                "root",
                ""
            );
            PreparedStatement ps = con.prepareStatement(
                "select * from customer where custno=?"
            );
            ps.setInt(1, Integer.parseInt(custno));
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                out.println("<h2>Customer Details</h2>");
                out.println("Customer No: " + rs.getInt(1) + "<br>");
                out.println("Name: " + rs.getString(2) + "<br>");
                out.println("City: " + rs.getString(3) + "<br>");
                out.println("Balance: " + rs.getDouble(4) + "<br>");
            } else {
                out.println("<h3 style='color:red'>Customer Not Found!</h3>");
            }
            con.close();
        } catch (Exception e) {
            out.println("Error: " + e.getMessage());
        }
    }
}
```

(C:\Java-II\apache-tomcat-9.0.115\webapps\CustomerSearch\WEB-INF\web.xml)

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         version="3.1">
```

```
    <servlet>
        <servlet-name>SearchServlet</servlet-name>
        <servlet-class>SearchServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>SearchServlet</servlet-name>
        <url-pattern>/search</url-pattern>
    </servlet-mapping>
</web-app>
```

Q31: Design an HTML page containing option buttons (Maths, Physics, Chemistry and Biology) and buttons submit and reset. When the user clicks submit, the server responds by adding a cookie containing the selected subject and sends a message back to the client. Program should not allow duplicate cookies to be written.

(C:\Java-II\apache-tomcat-9.0.115\webapps\SubjectCookie\subject.html)

```
<html>
<body>
<h2>Select Your Favourite Subject</h2>
<form action="subject" method="post">
<input type="radio" name="sub" value="Maths"> Maths <br>
<input type="radio" name="sub" value="Physics"> Physics <br>
<input type="radio" name="sub" value="Chemistry"> Chemistry <br>
<input type="radio" name="sub" value="Biology"> Biology <br><br>
<input type="submit" value="Submit">
<input type="reset" value="Reset">
</form>
</body>
</html>
```

(C:\Java-II\apache-tomcat-9.0.115\webapps\SubjectCookie\WEB-INF\web.xml)

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         version="3.1">
    <servlet>
        <servlet-name>SubjectServlet</servlet-name>
        <servlet-class>SubjectServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>SubjectServlet</servlet-name>
        <url-pattern>/subject</url-pattern>
    </servlet-mapping>
</web-app>
```

(C:\Java-II\apache-tomcat-9.0.115\webapps\SubjectCookie\WEB-INF\classes\SubjectServlet.java)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class SubjectServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String subject = request.getParameter("sub");
        if (subject == null) {
            out.println("<h3>Please select a subject!</h3>");
            return;
        }
        boolean alreadyExists = false;
        Cookie[] cookies = request.getCookies();
        if (cookies != null) {
            for (Cookie c : cookies) {
                if (c.getName().equals("subject") &&
                    c.getValue().equals(subject)) {
                    alreadyExists = true;
                }
            }
        }
        if (!alreadyExists) {
            Cookie ck = new Cookie("subject", subject);
            ck.setMaxAge(60 * 60); // 1 hour
            response.addCookie(ck);
            out.println("<h3>Cookie Added Successfully!</h3>");
        } else {
            out.println("<h3 style='color:red'>Duplicate Cookie Not Allowed!</h3>");
        }
        out.println("Selected Subject: " + subject);
    }
}
```

Q32: Write a JSP program to display the details of PATIENT (PatientNo, PatientName, PatientAddress, Patientage,PatientDiease) in tabular form on browser.

(C:\Java-II\apache-tomcat-9.0.115\webapps\PatientApp\patient.jsp)

```
<%@ page language="java" %>
<html>
<head>
    <title>Patient Details</title>
    <style>
        table {
```

```
            border-collapse: collapse;
            width: 80%;
        }
        th, td {
            border: 1px solid black;
            padding: 8px;
            text-align: center;
        }
        th {
            background-color: lightblue;
        }
    </style>
</head>
<body>
<h2>Patient Details</h2>
<%
    // Sample patient data
    String[][] patients = {
        {"101", "Nilesh", "Pune", "25", "Fever"},
        {"102", "Rahul", "Mumbai", "30", "Diabetes"},
        {"103", "Sneha", "Nashik", "22", "Asthma"},
        {"104", "Amit", "Delhi", "40", "Flu"}
    };
%>
<table>
    <tr>
        <th>Patient No</th>
        <th>Patient Name</th>
        <th>Patient Address</th>
        <th>Patient Age</th>
        <th>Patient Disease</th>
    </tr>
<%
    for(int i = 0; i < patients.length; i++) {
%>
    <tr>
        <td><%= patients[i][0] %></td>
        <td><%= patients[i][1] %></td>
        <td><%= patients[i][2] %></td>
        <td><%= patients[i][3] %></td>
        <td><%= patients[i][4] %></td>
    </tr>
<%
    }
%>
```

```
</table>
</body>
</html>
```

Q33: Create a JSP page for an online multiple choice test. The questions are randomly selected from a database and displayed on the screen. The choices are displayed using radio buttons. When the user clicks on next, the next question is displayed. When the user clicks on submit, display the total score on the screen.

(C:\Java-II\apache-tomcat-9.0.115\webapps\OnlineTest\exam.jsp)

```
<%@ page import="java.sql.*" %>
<%@ page session="true" %>
<%
    String action = request.getParameter("action");
    Integer score = (Integer) session.getAttribute("score");
    Integer count = (Integer) session.getAttribute("count");
    if(score == null){
        score = 0;
        count = 0;
    }
    if("next".equals(action) || "submit".equals(action)){
        String correct = (String) session.getAttribute("correctAns");
        String selected = request.getParameter("answer");
        if(selected != null && selected.equals(correct)){
            score++;
        }
        count++;
    }
    session.setAttribute("score", score);
    session.setAttribute("count", count);
    if("submit".equals(action)){
%>
<h2>Test Completed!</h2>
<h3>Your Total Score: <%= score %></h3>
<%
        session.invalidate();
        return;
    }
    // DB Connection
    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/examdb",
        "root",
        ""   // XAMPP default password empty
    );
```

```
    Statement st = con.createStatement();
    ResultSet rs = st.executeQuery(
        "SELECT * FROM question ORDER BY RAND() LIMIT 1"
    );
    if(rs.next()){
        session.setAttribute("correctAns", rs.getString("answer"));
%>
<h2>Online Multiple Choice Test</h2>
<form method="post">
<p><b>Q<%= count+1 %>. <%= rs.getString("question") %></b></p>
<input type="radio" name="answer" value="<%= rs.getString("opt1”) %>">
<%= rs.getString("opt1") %><br>
<input type="radio" name="answer" value="<%= rs.getString("opt2”) %>">
<%= rs.getString("opt2") %><br>
<input type="radio" name="answer" value="<%= rs.getString("opt3”) %>">
<%= rs.getString("opt3") %><br>
<input type="radio" name="answer" value="<%= rs.getString("opt4”) %>">
<%= rs.getString("opt4") %><br><br>
<input type="submit" name="action" value="next">
<input type="submit" name="action" value="submit">
</form>
<%
    }
    con.close();
%>
```

Q34: Consider the following entities and their relationships Movie (movie_no, movie_name, release_year) Actor(actor_no, name) Relationship between movie and actor is many – many with attribute rate in Rs. Create a RDB in 3 NF answer the following: a) Accept an actor name and display all movie names in which he has acted along with his name on top. b) Accept a movie name and list all actors in that movie along with the movie name on top.

```
CREATE DATABASE MovieDB;

USE MovieDB;

CREATE TABLE Movie (

   movie_no INT PRIMARY KEY,

   movie_name VARCHAR(50),

   release_year INT

);

 CREATE TABLE Actor (
```

```
    actor_no INT PRIMARY KEY,
    name VARCHAR(50)
);
 CREATE TABLE CastDetails (
    movie_no INT,
    actor_no INT,
    rate INT,
    PRIMARY KEY (movie_no, actor_no),
    FOREIGN KEY (movie_no) REFERENCES Movie(movie_no),
    FOREIGN KEY (actor_no) REFERENCES Actor(actor_no)
);
INSERT INTO Movie VALUES
(1,'KGF',2018),
(2,'RRR',2022),
(3,'Pushpa',2021);
 INSERT INTO Actor VALUES
(101,'Yash'),
(102,'Ram Charan'),
(103,'Allu Arjun');
 INSERT INTO CastDetails VALUES
(1,101,5000000),
(2,102,6000000),
(3,103,5500000),
(2,103,3000000);
-------------------------------------------------------------------------------
SELECT A.name AS Actor_Name, M.movie_name
FROM Actor A
```

JOIN CastDetails C ON A.actor_no = C.actor_no

JOIN Movie M ON C.movie_no = M.movie_no

WHERE A.name = 'Allu Arjun';

SELECT M.movie_name AS Movie_Name, A.name

FROM Movie M

JOIN CastDetails C ON M.movie_no = C.movie_no

JOIN Actor A ON C.actor_no = A.actor_no

WHERE M.movie_name = 'RRR';

Assignment 5

Q35: Create a Spring core example to display the message “If you can't explain it simply, you don't understand it well enough”.

(C:\Java-II\Ass5\SpringCoreExample\src\applicationContext.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="messageBean" class="Message">
        <property name="text"
         value="If you can't explain it simply, you don't understand it well enough"/>
    </bean>
</beans>
```

(C:\Java-II\Ass5\SpringCoreExample\src\MainApp.java)

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class MainApp {
    public static void main(String[] args) {
        ApplicationContext context =
                new ClassPathXmlApplicationContext("applicationContext.xml");
        Message msg = (Message) context.getBean("messageBean");
        msg.displayMessage();
    }
}
```

(C:\Java-II\Ass5\SpringCoreExample\src\Message.java)

```
public class Message {
    private String text;
    // Setter method for Dependency Injection
    public void setText(String text) {
        this.text = text;
    }
    public void displayMessage() {
        System.out.println(text);
    }
}
```

Q36: Write a program to display the Current Date using spring.

(C:\Java-II\Ass5\SpringCurrentDate\src\applicationContext.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="dateBean" class="DateBean"/>
</beans>
```

(C:\Java-II\Ass5\SpringCurrentDate\src\DateBean.java)

```
import java.util.Date;
public class DateBean {
    public Date getCurrentDate() {
        return new Date();
    }
}
```

(C:\Java-II\Ass5\SpringCurrentDate\src\MainApp.java)

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class MainApp {
    public static void main(String[] args) {
        ApplicationContext context =
                new ClassPathXmlApplicationContext("applicationContext.xml");
        DateBean obj = (DateBean) context.getBean("dateBean");
        System.out.println("Current Date: " + obj.getCurrentDate());
    }
}
```

Q37: Design simple student information like Student_id, Student_Name and Student_Age using Spring Framework.

(C:\Java-II\Ass5\SpringStudent\src\applicationContext.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="studentBean" class="Student">
        <property name="student_id" value="101"/>
        <property name="student_name" value="Nilesh"/>
        <property name="student_age" value="21"/>
    </bean>
</beans>
```

(C:\Java-II\Ass5\SpringStudent\src\MainApp.java)

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class MainApp {
    public static void main(String[] args) {
        ApplicationContext context =
                new ClassPathXmlApplicationContext("applicationContext.xml");
        Student s = (Student) context.getBean("studentBean");
        s.display();
    }
}
```

(C:\Java-II\Ass5\SpringStudent\src\Student.java)

```
public class Student {
    private int student_id;
    private String student_name;
    private int student_age;
    // Setter methods (Dependency Injection)
    public void setStudent_id(int student_id) {
        this.student_id = student_id;
    }
    public void setStudent_name(String student_name) {
        this.student_name = student_name;
    }
    public void setStudent_age(int student_age) {
        this.student_age = student_age;
```

```
    }
    public void display() {
        System.out.println("Student ID   : " + student_id);
        System.out.println("Student Name : " + student_name);
        System.out.println("Student Age  : " + student_age);
    }
}
```

Q38: Design the Employee login form application using spring form MVC validation.

(C:\Java-II\Ass5\EmployeeLogin\pom.xml)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>EmployeeLogin</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>war</packaging>
    <dependencies>
        <!-- Spring MVC -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-webmvc</artifactId>
            <version>5.3.27</version>
        </dependency>
        <!-- Validation API -->
        <dependency>
            <groupId>jakarta.validation</groupId>
            <artifactId>jakarta.validation-api</artifactId>
            <version>3.0.2</version>
        </dependency>
        <!-- JSTL -->
        <dependency>
            <groupId>jakarta.servlet.jsp.jstl</groupId>
            <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
            <version>2.0.0</version>
        </dependency>
        <dependency>
            <groupId>org.glassfish.web</groupId>
            <artifactId>jakarta.servlet.jsp.jstl</artifactId>
            <version>2.0.0</version>
        </dependency>
        <!-- Servlet API -->
```

```
        <dependency>
            <groupId>jakarta.servlet</groupId>
            <artifactId>jakarta.servlet-api</artifactId>
            <version>5.0.0</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>
    <build>
        <finalName>EmployeeLogin</finalName>
    </build>
</project>
```

(C:\Java-II\Ass5\EmployeeLogin\src\main\java\com\example\config\WebConfig.java)

```
package com.example.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.view.InternalResourceViewResolver;
import org.springframework.validation.beanvalidation.LocalValidatorFactoryBean;
@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "com.example")
public class WebConfig {
    @Bean
    public InternalResourceViewResolver viewResolver() {
        InternalResourceViewResolver resolver =
                new InternalResourceViewResolver();
        resolver.setPrefix("/WEB-INF/views/");
        resolver.setSuffix(".jsp");
        return resolver;
    }
    @Bean
    public LocalValidatorFactoryBean validator() {
        return new LocalValidatorFactoryBean();
    }
}
```

(C:\Java-II\Ass5\EmployeeLogin\src\main\java\com\example\controller\EmployeeController.java)

```
@Controller
public class EmployeeController {
    @RequestMapping(value = "/login", method = RequestMethod.GET)
    public String showForm(Model model) {
        model.addAttribute("employee", new Employee());
```

```
        return "login";
    }
    @RequestMapping(value = "/login", method = RequestMethod.POST)
    public String submitForm(@Valid @ModelAttribute("employee") Employee employee,
                             BindingResult result, Model model) {
        if (result.hasErrors()) {
            return "login";
        }
        if ("admin".equals(employee.getUsername()) && "1234".equals(employee.getPassword()))
{
            model.addAttribute("msg", "Login Successful!");
            return "success";
        } else {
            model.addAttribute("msg", "Invalid Credentials!");
            return "login";
        }
    }
}
```

(C:\Java-II\Ass5\EmployeeLogin\src\main\java\com\example\model\Employee.java)

```
package com.example.model;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.Size;
public class Employee {
    @NotEmpty(message = "Username is required")
    private String username;
    @NotEmpty(message = "Password is required")
    @Size(min = 4, max = 12, message = "Password must be 4-12 characters")
    private String password;
    // Getters and Setters
    public String getUsername() {
        return username;
    }
    public void setUsername(String username) {
        this.username = username;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}
```

(C:\Java-II\Ass5\EmployeeLogin\src\main\webapp\WEB-INF\spring-servlet.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="https://www.springframework.org/schema/context"
       xmlns:mvc="https://www.springframework.org/schema/mvc"
       xsi:schemaLocation="
           https://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd
           https://www.springframework.org/schema/context
https://www.springframework.org/schema/context/spring-context.xsd
           https://www.springframework.org/schema/mvc
https://www.springframework.org/schema/mvc/spring-mvc.xsd">
    <mvc:annotation-driven/>
    <context:component-scan base-package="com.example"/>
    <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/views/"/>
    <property name="suffix" value=".jsp"/>
    </bean>
</beans>
```

(C:\Java-II\Ass5\EmployeeLogin\src\main\webapp\WEB-INF\web.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee" version="5.0">
   <servlet>
    <servlet-name>spring</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name>spring</servlet-name>
        <url-pattern>/</url-pattern>
    </servlet-mapping>
</web-app>
```

(C:\Java-II\Ass5\EmployeeLogin\src\main\webapp\WEB-INF\views\login.jsp)

```
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>
<html>
<head>
    <title>Employee Login</title>
</head>
<body>
<h2>Employee Login Form</h2>
```

```
<form:form method="POST" modelAttribute="employee">
    <table>
        <tr>
            <td>Username:</td>
            <td><form:input path="username"/></td>
            <td><form:errors path="username" cssClass="error"/></td>
        </tr>
        <tr>
            <td>Password:</td>
            <td><form:password path="password"/></td>
            <td><form:errors path="password" cssClass="error"/></td>
        </tr>
        <tr>
            <td colspan="2"><input type="submit" value="Login"/></td>
        </tr>
    </table>
</form:form>
<c:if test="${not empty msg}">
    <p style="color:red">${msg}</p>
</c:if>
</body>
</html>
```

(C:\Java-II\Ass5\EmployeeLogin\src\main\webapp\WEB-INF\views\success.jsp)

```
<html>
<head>
    <title>Login Success</title>
</head>
<body>
<h2>${msg}</h2>
</body>
</html>
```